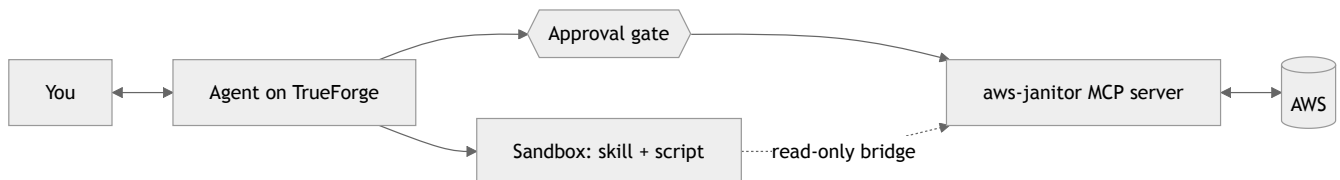


Cloud Cost Janitor

The problem. Every AWS account collects leftovers: an instance nobody logs into, volumes that outlived their server, a load balancer with nothing behind it. Each costs a few dollars a month, so nobody bothers, and they bill forever. Cleaning up by hand means three consoles, CloudWatch graphs, and a nervous delete.

The solution. An agent on TrueForge that finds this waste, prices it, and removes it, but only after a person clicks Allow.



How it works. The agent is a short definition: role, two MCP connectors, one skill. The skill is the playbook, versioned in git and cloned into the sandbox at a pinned tag. The MCP server is Python, talking to one AWS account with an identity named explicitly in `.env`.

You ask for an audit. The server scans EC2, EBS and ELBv2, pulls 14 days of CloudWatch utilisation, and applies AWS Trusted Advisor's thresholds, so "idle" means CPU under 10% and network under 5 MB a day. Prices come from the Price List API. The agent then runs a script in the sandbox to total the plan, so the numbers come from code, not from the model's arithmetic. You get a table: resource, evidence, confidence, dollars per month, with protected resources marked off limits.

The safety chain. Nine tools are read-only. One, `delete_resource`, is destructive, and TrueForge pauses on every call. On Allow, the server checks the plan is still live and the resource still in it, refuses protected tags, re-reads the resource from AWS, snapshots any volume, and only then deletes. Denials are terminal. Everything lands in an audit log that never holds identity or credentials, and AWS error text is redacted before the model sees it.

Stack. TrueForge, Kimi K3, Python, FastMCP, boto3.